# Bézier Curve Series

Code Analysis and Design Review

Scripts 01–35: Architecture, Patterns, Trade-offs, and Julia Lessons

E.R. Nurwijayadi

2026-03-15

*Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.*

This document steps back from individual script explanations and analyses the series as a whole. It covers the architectural progression across all 15 scripts, recurring code patterns, deliberate design decisions and their trade-offs, Julia-specific idioms introduced along the way, and the curvature thread that connects the later scripts.

---

## Contents

**1 Part 1: Architecture of the Series** **3**
1.1 The Three-Axis Progression . . . 3
1.2 Script Map: All 15 Scripts on Three Axes . . . 3
1.3 How the Series Teaches . . . 4

**2 Part 2: Recurring Code Patterns** **4**
2.1 The Universal Curve Matrix Builder . . . 4
2.2 The `CASE` Switch Pattern . . . 5
2.3 The `is_3d` Branch Pattern . . . 5
2.4 The `while` De Casteljau Loop . . . 6
2.5 Colour-with-Alpha Tuples . . . 6
2.6 Legend Control: Three Strategies . . . 7

**3 Part 3: Design Decisions and Trade-offs** **7**
3.1 Struct Design: How It Evolved . . . 7
3.2 Arithmetic Choice: Float64 vs Rational{Int} . . . 8
3.3 CairoMakie vs GLMakie . . . 8
3.4 Named Functions vs Inline: The De Casteljau Progression . . . 9

3.5 Symbolic vs Numeric Evaluation . . . . . 9

**4 Part 4: Julia-Specific Lessons 10**

4.1 Lesson Map: Which Script Teaches What . . . . . 10
4.2 Broadcast: The Unifying Operator . . . . . 10
4.3 Multiple Dispatch: A Concrete Example . . . . . 11
4.4 The `let` Closure Problem . . . . . 11
4.5 `Observable` vs `Ref`: Two Kinds of State . . . . . 12

**5 Part 5: The Curvature Thread 12**

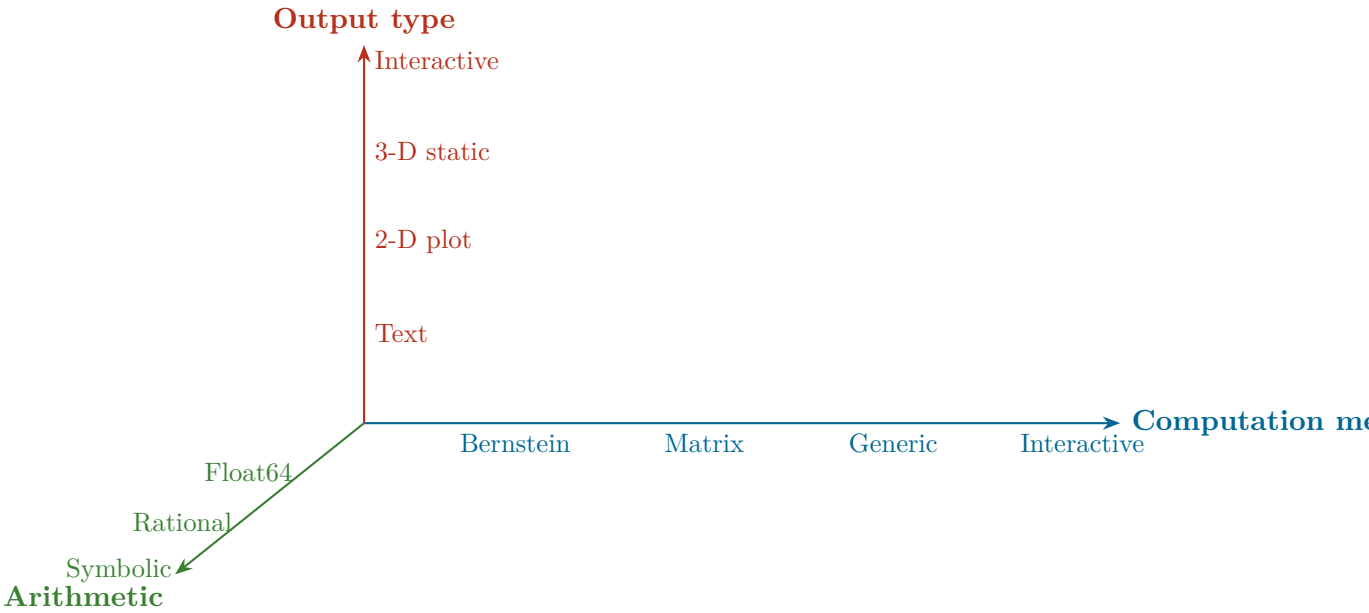
5.1 How Curvature Connects the Series . . . . . 12
5.2 The Derivative Control Point Insight . . . . . 13
5.3 2-D vs 3-D Curvature: One Formula, Two Implementations . . . . . 13
5.4 The Curvature Comb as a Design Verification Tool . . . . . 13

**6 Summary: What the Series Teaches About Software Design 14**

# 1. Part 1: Architecture of the Series

## 1.1. The Three-Axis Progression

The 15 scripts are not random — they form a deliberate path through a three-dimensional design space. Each axis is varied independently, so the reader can isolate one change at a time.




## 1.2. Script Map: All 15 Scripts on Three Axes

| Script | Computation | Arithmetic | Output | New concept |
|---|---|---|---|---|
| 01 | Bernstein (hand) | Float64 | Text | `Poly` type, operator overloading |
| 02 | Bernstein | Rational | Text | Symbolics.jl, `@variables` |
| 03 | Matrix (pedagogical) | Float64 | Text | $U \cdot M \cdot G$, derive_M |
| 04 | Matrix (straight) | Float64 | Text | Section-based output |
| 05 | Matrix | Rational | Text | `fmt_poly` bridging types |
| 11p | Bernstein | Float64 | 2-D plot | Plots.jl, `hcat+splat+transpose` |
| 11m | Bernstein | Float64 | 2-D plot | CairoMakie, Figure/Axis split |
| 12p | De Casteljau | Float64 | 2-D plot | `lerp`, named tuple return |
| 12m | De Casteljau | Float64 | 2-D plot | `titlegap`, no `label` = hidden |
| 21 | Bernstein | Float64 | 3-D static | `Axis3`, elevation/azimuth |
| 22 | De Casteljau | Float64 | 3-D static | `getfield`, NaN dummy lines |
| 31 | Generic Bernstein | Float64 | 2-D/3-D | `is_3d` branch, `while` DC |
| 32 | Generic Symbolic | Mixed | 2-D/3-D | `build_symbolic_poly` |
| 34 | Generic + Curvature | Float64 | 2-D/3-D | `d1_pts`/`d2_pts`, `draw_comb!` |
| 35 | Interactive | Float64 | UI | `Observable`, `Toggle`, `Textbox` |

### 1.3. How the Series Teaches

**Each script introduces exactly one new idea.**

Scripts 01–05 vary only the *arithmetic* (Float64 vs Rational) and the *method* (Bernstein vs matrix), keeping the output as plain terminal text. This isolates the mathematics from all plotting concerns.

Scripts 11–12 introduce plotting for the first time, but use the simplest possible curve (quadratic 2-D) so the new plotting API is not competing with new math.

Scripts 21–22 add 3-D, but reuse the quartic curve already understood from scripts 01–05.

Scripts 31–35 generalise everything, stacking interactivity on top of the generic architecture one layer at a time.

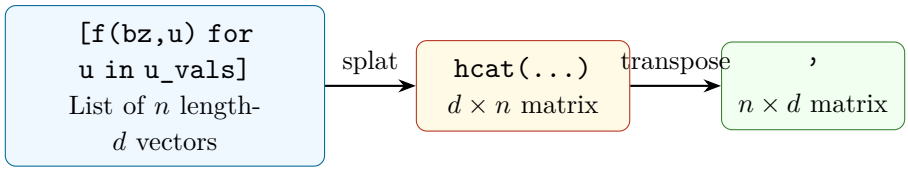
A beginner can follow each step without being overwhelmed because the foundation is always already familiar.

## 2. Part 2: Recurring Code Patterns

### 2.1. The Universal Curve Matrix Builder

The same three-step idiom appears in *every* plotting script from 11 onward:

```
hcat([f(bz, u) for u in u_vals]...)'
```




| Script | Function | Result shape |
|---|---|---|
| 11 | `curve(bz)` | $200 \times 2$ |
| 12 | `dc_curve(dc)` | $200 \times 2$ |
| 21 | `bq_curve(bz)` | $300 \times 3$ |
| 31 | `bz_curve(bz)` | $300 \times$ dim |
| 34 | `bz_curve(pts)` | $300 \times$ dim |
| 35 | `bz_curve(pts)` | $300 \times$ dim |

The pattern is identical in every script. Once a reader understands it in script 11, they recognise it immediately in all subsequent scripts.

## 2.2. The CASE Switch Pattern

```
const CASE = 2   # only change this line

const CASE_DATA = Dict(
    1 => ( points = ..., output = "...", ),
    2 => ( points = ..., output = "...", ),
)

function main()
    data = CASE_DATA[CASE]
    ...
end
```

This pattern appears in scripts 01–05, 31, 32, 34. It separates the *configuration* (what case to run) from the *logic* (how to run it). The single line `const CASE = N` is the only line a user needs to change.

| What varies between cases | What stays constant |
|---|---|
| Control point matrix (shape and values) | All math functions |
| Number of dimensions (2 or 3) | All drawing functions |
| Output filename | The `CASE` switch mechanism |
| $u$ highlight values | `main()` structure |

## 2.3. The is_3d Branch Pattern

Scripts 31, 32, 34, 35 all contain blocks of the form:

```
if is_3d
    lines!(ax, xs, ys, zs, ...)
else
    lines!(ax, xs, ys, ...)
end
```

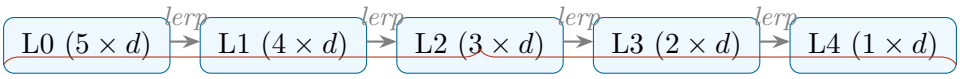
This appears for every drawing call: `lines!`, `scatter!`, `text!`, axis creation, limit setting. The duplication is deliberate — it keeps each branch self-contained and easy to read, even though it is verbose.

An alternative would be to pass coordinates as a tuple and splat them: `lines!(ax, coords...)`. This would halve the number of lines but make each line harder to read. The series chose readability over compactness.

## 2.4. The `while` De Casteljau Loop

```
levels  = Matrix{Float64}[copy(pts)]
current = copy(pts)
while size(current, 1) > 1
    n       = size(current, 1)
    current = vcat([((1-u).*current[j,:] .+ u.*current[j+1,:])' for j in
    ↪  1:n-1]...)
    push!(levels, current)
end
```

This exact pattern appears in scripts 31, 32, 34, 35. It replaces the explicit level-by-level functions from scripts 12 and 22 (`reduce_level` called four times). The `while` version works for any degree automatically.




**while loop — one iteration per level**

## 2.5. Colour-with-Alpha Tuples

Makie accepts colours as `(name, alpha)` pairs. This pattern appears in every Makie script from script 22 onward:

| Usage | Code | Meaning |
|---|---|---|
| Construction line | `(col, 0.75)` | 75% opacity |
| Connector line | `(col, 0.35)` | 35% — very faint |
| Comb spike | `(:slategray, 0.5)` | 50% |
| Comb envelope | `(:slategray, 0.4)` | 40% |
| Legend frame | `(:black, 0.3)` | 30% |
| Legend background | `(:white, 0.8)` | 80% |

The pattern lets the reader layer visual information: full-opacity elements (curve, markers) are foreground; semi-transparent elements (construction lines, comb) are background.

### 2.6. Legend Control: Three Strategies

Across the scripts, three different strategies are used to control which elements appear in the legend:

| Strategy | Code | When used |
|---|---|---|
| Include | `label = "name"` | Main elements: curve, polygon, highlighted points |
| Exclude (Plots.jl) | `label = false` | Auxiliary scatter dots (De Casteljau levels) |
| Exclude (Makie) | (omit `label`) | Same effect in Makie — no keyword = no entry |
| Placeholder | `label = ""` | Script 31: dot-only legend entries for level colours |
| NaN dummy | `lines!([NaN],[NaN],[NaN], label=lbl)` | Script 22: style entries (solid/dashed/dotted) |

> **NaN as an invisible marker.** The NaN dummy line pattern (script 22) is a professional trick used in real scientific plotting code. It lets you add a legend entry that explains a visual convention (line style, colour) without drawing anything visible in the plot area.

## 3. Part 3: Design Decisions and Trade-offs

### 3.1. Struct Design: How It Evolved

The struct holding control points changed four times across the series, each time for a clear reason:

| Scripts | Struct | Fields | Why |
|---|---|---|---|
| 11–12 | `BezierQuadratic` | `p0, p1, p2 ::Vector` | 3 named fields are readable; small fixed degree |
| 21 | `BezierQuartic` | `points ::Matrix` | 5 fields would be clutter; matrix is cleaner |
| 31–33 | `BezierCurve` | `points, n, degree, dim` | Generic: infers all metadata from matrix shape |
| 34–35 | `BezierCurve` (extended) | + `d1_pts, d2_pts` | Curvature needs pre-computed derivative control points |

**Named fields vs matrix field:**

`BezierQuadratic` with `p0, p1, p2` is readable in the point function:

```
b0 .* bz.p0 .+ b1 .* bz.p1 .+ b2 .* bz.p2
```

But for 5 points it becomes:

```
b0.*bz.p0 .+ b1.*bz.p1 .+ b2.*bz.p2 .+ b3.*bz.p3 .+ b4.*bz.p4
```

The generic `sum(bernstein(...)  .* bz.points[i,:]  for i ...)` is shorter and works for any degree. The trade-off: named fields are self-documenting; indexed access is compact but requires knowing the convention.

### 3.2. Arithmetic Choice: Float64 vs Rational{Int}

| Type | Strengths | Weaknesses |
|---|---|---|
| `Float64` | Fast; compatible with all plotting; natural for most users | Rounding error accumulates; `rationalize()` needed to display fractions |
| `Rational{Int}` | Exact; fractions display naturally as `269//256`; no rounding error | Slower; denominators grow; needs conversion to Float64 for display |

Scripts 02 and 05 use `Rational{Int}` to demonstrate that the exact fractions computed algebraically (scripts 01–05) genuinely match the plotted values. The rest of the series uses `Float64` because plotting coordinates are inherently approximate.

### 3.3. CairoMakie vs GLMakie

| Backend | Used in | Reason |
|---|---|---|
| `CairoMakie` | Scripts 11m, 12m, 21, 22 | Static PNG output; no display server needed |
| `GLMakie` | Scripts 31, 32, 34, 35 | Interactive window; 3-D rotation with mouse |

The switch from CairoMakie to GLMakie at script 31 coincides with the introduction of the generic `is_3d` architecture. Once a script could produce either a 2-D or 3-D plot, interactivity became more valuable — and GLMakie's mouse-driven 3-D rotation makes the quartic 3-D case much easier to explore.

### 3.4. Named Functions vs Inline: The De Casteljau Progression

| Scripts | Approach | Trade-off |
|---|---|---|
| 12 | `lerp` + `steps()` named tuple | Clean, readable, but fixed at quadratic |
| 22 | `lerp3` + `reduce_level` + `dc_steps` | More general (any level count) but still fixed degree |
| 31–35 | Inline `while` loop, no separate functions | Maximally general; self-contained; harder to unit-test |

The progression shows a deliberate move from *pedagogical clarity* (named function per concept) toward *engineering compactness* (one generic loop). Both have value — the individual function approach is better for teaching; the loop is better for production.

### 3.5. Symbolic vs Numeric Evaluation

Scripts 31 and 32 produce identical plots from visually identical code. The difference is entirely in how `bz_point` works:

| Script | Method | Suitable when |
|---|---|---|
| 31 | Direct Bernstein: $\sum B(n, i, u) \cdot P_i$ per call | You want speed and simplicity |
| 32 | Build polynomial once; substitute $u$ for each evaluation | You want to inspect, differentiate, or print the polynomial itself |

Script 32's `main()` prints the expanded polynomials to the terminal before drawing — something script 31 cannot do. This connects the plotting scripts back to the algebraic results of scripts 01–05, closing a loop in the series.

# 4. Part 4: Julia-Specific Lessons

## 4.1. Lesson Map: Which Script Teaches What

| Julia concept | What it is | First seen |
|---|---|---|
| Multiple dispatch | Same function name, different behaviour by type | Script 01 (`fmt_rat`) |
| Struct + outer constructor | Derived fields computed in constructor | Scripts 01, 11 |
| Broadcast (`.* .+`) | Element-wise operation on any array shape | Script 11 |
| Named tuple return | `return (field1=v1, field2=v2)` | Script 12 |
| `sum(generator; init=)` | Accumulate a computed series with a type-providing seed | Script 05 |
| `Union{T, Nothing}` | Optional typed field; checked with `isnothing` | Script 34 |
| `diff(A, dims=1)` | Row-difference shortcut for derivative control points | Script 34 |
| `getfield(obj, :sym)` | Dynamic field access by symbol variable | Script 22 |
| `let` closure capture | Freeze loop variable values for callbacks | Script 35 |
| `Observable` / `Ref` | Notifying vs non-notifying mutable cell | Script 35 |
| `on(obs) do ... end` | Reactive callback registration | Script 35 |
| `tryparse(T, s)` | Safe string-to-type conversion, returns `nothing` on failure | Script 35 |

## 4.2. Broadcast: The Unifying Operator

The `.` broadcast operator appears in every script. It is the reason the same formula works for both scalar-times-vector and vector-times-vector:

```
# Script 11: scalar weight * 2-D point
b0 .* bz.p0              # Float64 .* Vector{Float64}

# Script 34: curvature * 3-D normal vector
scale * kap[i] .* N      # Float64 .* Vector{Float64}
```

```
# Script 34: diff control points (row-wise subtraction)
deg .* diff(points, dims=1)    # scalar .* Matrix
```

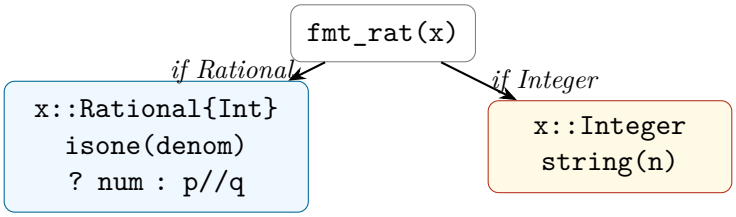
In all cases the formula is written identically regardless of dimension. A 2-D vector and a 3-D vector are handled by the same code without any `if dim == 2` branching.

### 4.3. Multiple Dispatch: A Concrete Example

Script 01 introduced `fmt_rat` as two methods on two types:

```
fmt_rat(r::Rational{Int}) = isone(denominator(r)) ?
    string(numerator(r)) : string(r)
fmt_rat(n::Integer)       = string(n)
```

This is Julia's multiple dispatch in its simplest form. No `if typeof(x) == ...` checking — the language dispatches automatically based on the runtime type.




### 4.4. The `let` Closure Problem

The `let` fix in script 35 addresses one of the most common bugs in reactive/callback-heavy code in any language:

| Without `let` (broken) | With `let` (correct) |
|---|---|
| `for i in 1:3`<br>`  for d in 1:2`<br>`  on(tb.stored_string) do val`<br>`  # i and d are shared`<br>`  # captured by reference`<br>`  new_pts[i, d] = v  # ↪ WRONG`<br>`  end`<br>`  end`<br>`end`<br>`# After loop: i=3, d=2`<br>`# ALL callbacks update [3,2]` | `for i in 1:3`<br>`  for d in 1:2`<br>`  let row=i, col=d`<br>`  on(tb.stored_string) do ↪ val`<br>`  # row and col are local`<br>`  # captured by value`<br>`  new_pts[row, col] = v`<br>`  end`<br>`  end  # each callback has`<br>`  end  # its own row, col`<br>`end` |

**This pattern matters far beyond Julia.** The same closure-capture bug exists in JavaScript (`var` in loops), Python (lambdas in loops), and C# (delegates in foreach). The `let` solution in Julia is the equivalent of JavaScript's `let` (block scope), Python's default-argument trick, or C#'s loop variable copy. Once you understand it in one language, you recognise it everywhere.

### 4.5. `Observable` vs `Ref`: Two Kinds of State

Script 35 uses both, and the choice between them matters:

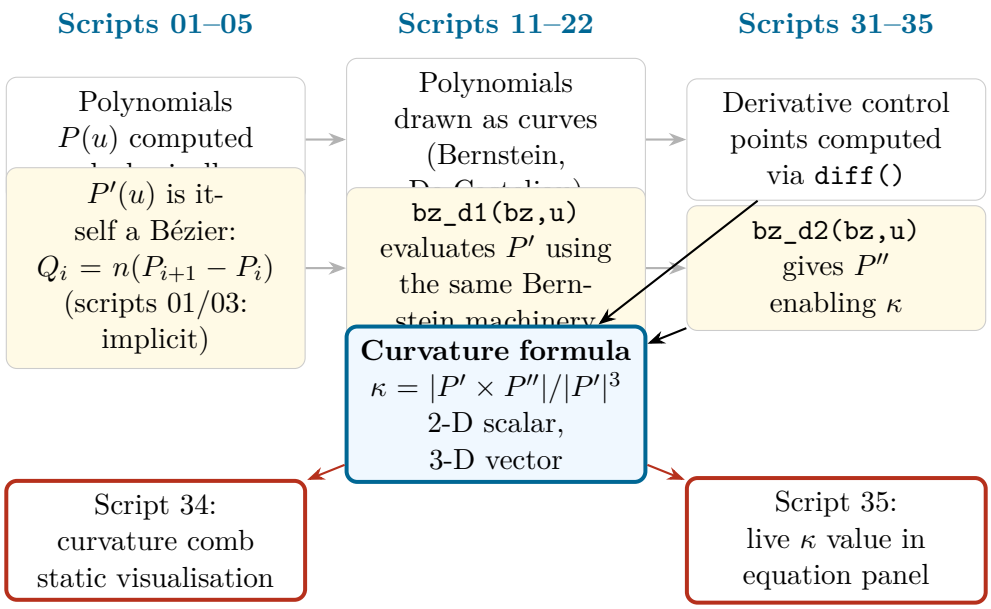
| | `Ref(x)` | `Observable(x)` |
|---|---|---|
| Read | `ref[]` | `obs[]` |
| Write | `ref[] = v` | `obs[] = v` |
| Notifies listeners? | No | Yes |
| Used for | `active_case`, `switching`, `ax_ref`, `legend_ref` | `pts_obs`, `u_obs`, `tog_poly.active` |
| Callback? | None | `on(obs) do ... end` |

The design rule: use `Observable` when a change should automatically trigger a redraw. Use `Ref` for internal bookkeeping state that does not need to propagate. `switching` is a `Ref` precisely because it is a guard flag — if it were an `Observable`, setting it could itself trigger the redraw you are trying to suppress.

## 5. Part 5: The Curvature Thread

### 5.1. How Curvature Connects the Series

Curvature is not introduced until script 34, but the mathematical foundation is laid much earlier. The thread runs from the polynomial derivatives in scripts 01–05 all the way through to the live $\kappa$ value in the script 35 equation panel.



### 5.2. The Derivative Control Point Insight

The key insight that makes efficient curvature computation possible is that the derivative of a Bézier curve is itself a Bézier curve. This means `eval_poly` — the same function that evaluates the original curve — can also evaluate the derivative and second derivative. No symbolic differentiation is needed.

```
# All three use the same evaluator:
bz_point = eval_poly(bz.points,  u)   # P(u)
bz_d1    = eval_poly(bz.d1_pts,  u)   # P'(u)
bz_d2    = eval_poly(bz.d2_pts,  u)   # P''(u)
```

| Quantity | Control points | Degree | Result |
|---|---|---|---|
| $P(u)$ | `bz.points` $(n \times d)$ | $n-1$ | Point on curve |
| $P'(u)$ | `bz.d1_pts` $((n-1) \times d)$ | $n-2$ | Tangent vector |
| $P''(u)$ | `bz.d2_pts` $((n-2) \times d)$ | $n-3$ | Curvature direction |

### 5.3. 2-D vs 3-D Curvature: One Formula, Two Implementations

The curvature formula $\kappa = |P' \times P''|/|P'|^3$ is the same in both dimensions, but the cross product means different things:

| Dimension | Cross product $P' \times P''$ | Code |
|---|---|---|
| 2-D (the $z$-component of the 3-D cross product) | Scalar: $P'_x P''_y - P'_y P''_x$ | `abs(d1[1]*d2[2] - d1[2]*d2[1])` |
| 3-D | Vector of length $\lvert P'\rvert\lvert P''\rvert \sin\theta$ | `norm(cross(d1, d2))` |

The 2-D formula avoids the overhead of constructing a zero-padded 3-D vector just to call `cross`. The 3-D formula reuses the `LinearAlgebra.cross` function directly.

### 5.4. The Curvature Comb as a Design Verification Tool

Beyond being visually striking, the curvature comb in scripts 34–35 serves a practical purpose in curve design.

**What designers read from a curvature comb:**

**Uniform comb** (constant spike height) — the curve has constant curvature. A circle has a perfectly flat comb envelope.

**Smooth comb** (continuously varying height) — curvature changes gradually. This is desirable for fair curves in industrial design.

**Spiky or discontinuous comb** — curvature changes abruptly. At a kink (discontinuous tangent), the comb has a spike of infinite height. This is undesirable in automotive body panels, aerofoils, and other applications where visual smoothness matters.

> The quadratic Bézier in Case 1 produces a smooth, symmetric comb — confirming it is a fair curve. The quartic Bézier in Case 2 shows a more complex profile reflecting the tighter bends introduced by the off-axis control points $P_1$ and $P_2$.

## 6. Summary: What the Series Teaches About Software Design

Looking at the 15 scripts together, five software design principles emerge:

| Principle | How it appears | Scripts |
|---|---|---|
| **Separation of concerns** | Math, drawing, and UI are separate functions (`bz_point`, `draw_all!`, `redraw`) | 31–35 |
| **Single point of change** | `const CASE = N` controls everything | 01–05, 31–34 |
| **Generalise at the right time** | Quadratic first, then quartic, then generic — not generic from the start | 11→21→31 |
| **Reuse by composition** | `draw_all!` is called by both `draw_and_show` (scripts 34) and `redraw` (script 35) | 34–35 |
| **Fail safely** | `tryparse`, `isnothing` guards, `speed < 1e-10 && return 0.0` | 34–35 |

Code analysis for the Bézier series, scripts 01–35 | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated